



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

基于 MPI 任务池的矩阵 SVD 算子并行化实验

姓名：邹俊轩

学号：2412592

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

GitHub 仓库：[HTTPS://GITHUB.COM/POINTZU/MPI](https://github.com/POINTZU/MPI)

2026 年 6 月 9 日

摘要

本实验选择矩阵奇异值分解 (SVD) 算子作为 MPI 编程题目。将并行化集中在 Golub-Kahan (GKH) 迭代阶段: 0 号进程负责维护活动块任务池, 其余进程空闲后领取非平凡活动块, 执行一轮 bulge chasing, 并只回传对应的 B 子块以及 U, V 的相关列。实验在课程服务器上采用 `mpic++` 编译并通过 `qsub_mpi.sh` 提交, 共采集 41 组有效运行数据, 覆盖 MPI 进程数、编译优化等级、SIMD 开关和重复运行稳定性。全部原始输出均通过 5 个测试样例, 最大相对重构误差为 4.80×10^{-13} , 最大正交误差约为 2.38×10^{-13} 。

实验表明, 普通随机矩阵上的 SVD 并不随 MPI 进程数单调加速。以 GKH 阶段为例, $np = 4$ 时平均耗时从 23147.60 ms 降至 15975.63 ms, 加速比为 1.45; 继续增加到 $np = 8$ 、 $np = 16$ 后收益下降, $np = 32$ 反而退化为 0.34 倍。主要原因是 GKH 的活动块并行度不足, 且跨节点通信、矩阵广播和结果合并开销快速增大。SIMD 对上二对角化阶段有 1.26 倍收益, 但在 MPI+GKH 阶段几乎没有改善, 说明此时瓶颈已经由向量计算转向任务粒度和通信开销。本文据此给出基于实验数据的性能分析和后续改进方向。

关键词: SVD; MPI; Golub-Kahan; 任务池; SIMD; 性能分析

目录

1 实验目标与要求理解	1
1.1 选题要求	1
1.2 完成内容概述	1
2 算法与 MPI 并行设计	1
2.1 串行 SVD 流程	1
2.2 活动块并行粒度	1
2.3 主从式任务池	2
2.4 通信量设计	2
3 实现与代码分析	2
3.1 文件结构	2
3.2 矩阵连续内存接口	3
3.3 模式选择	3
3.4 任务分发与工作进程协议	3
3.5 结果打包与通信开销	4
3.6 统计字段与 profiling	4
4 实验方法	5
4.1 实验环境与数据采集	5
4.2 Profiling 设计	5
5 实验结果与分析	5
5.1 正确性验证	5
5.2 MPI 规模扩展	6
5.3 基于计时数据的开销分析	8
5.4 编译优化等级影响	8
5.5 SIMD 与 MPI 的组合	9
5.6 重复运行稳定性	10
5.7 本地 VTune 热点 profiling	11

6 进阶要求探索	12
6.1 不同平台与并行形态对比	12
6.2 不同 MPI 编程方法的比较	14
6.3 其他算法优化策略	15
7 综合讨论	15
7.1 为什么 $np = 4$ 最优	15
7.2 MPI 任务池的价值与限制	15
7.3 与前序 SIMD 实验的关系	16
8 总结	16
附录 A: 生成式 AI 辅助学习对话记录	17

1 实验目标与要求理解

1.1 选题要求

SVD 章节指出，多进程版本的难点在于：各进程地址空间独立，矩阵拷贝和通信开销很大，结果合并时竞争更严重，

手册对 SVD 提出了三个重点：第一，实现主从式结构，至少完成一个任务池；第二，围绕子矩阵或活动块组织迭代，避免已经收敛的 1×1 子问题反复参与计算；第三，完成加速和 profiling。

1.2 完成内容概述

本次实验在上一次 SVD 代码基础上完成如下工作：

- 在 `matrix.h` 中增加连续内存访问接口，使 MPI 能够广播矩阵数据。
- 在 `gkh.cpp` 中加入 `SVD_USE_MPI` 条件编译路径；使用 MPI 编译时，GKH 阶段默认进入 MPI 任务池模式。
- 设计 0 号进程调度、其他进程工作领取的主从式活动块任务池。只将长度大于 1 的非平凡活动块放入任务池，已经收敛的 1×1 块不再生成计算任务。
- 继承前序实验中的 SIMD 优化路径，并通过 `-DSVD_DISABLE_SIMD` 做对比实验。

2 算法与 MPI 并行设计

2.1 串行 SVD 流程

程序对 $m \times n$ 矩阵 A 计算 SVD。第一阶段使用左右 Householder 变换将矩阵化为上二对角矩阵：

$$A = UBV^T.$$

第二阶段在上二对角矩阵 B 上执行 Golub-Kahan 迭代，通过 Wilkinson shift 和 Givens 旋转进行 bulge chasing，使超对角元素逐步收敛到 0。最后程序将奇异值调整为非负并降序排列，得到标准化的 S 矩阵。该流程来自经典 SVD 算法 [1, 2]。

2.2 活动块并行粒度

GKH 迭代中，如果某个超对角元素已经足够接近 0，则上二对角矩阵可以被切分为多个相互独立的活动块。一个活动块 $[l, r]$ 表示第 l 到第 r 个奇异值相关的子问题尚未收敛。不同活动块之间不共享 B 的子块，更新 U, V 时也只涉及不重叠的列区间，因此可以在同一轮中并行处理。

不过，普通随机矩阵通常会在较长时间内保持一个大的活动块。此时即便启动多个 MPI 进程，也只有一个块可以推进，其余进程只能等待。只有当 deflation 产生多个非平凡活动块，或者输入矩阵本身具有块结构时，活动块任务池才有足够并行度。因此本实验的核心问题不是“MPI 是否一定加速”，而是通过数据判断：在本框架的端到端 SVD 测试中，活动块并行度是否足以抵消通信和复制开销。

2.3 主从式任务池

本实现采用 0 号进程作为调度者，其他进程作为工作进程。根进程扫描当前上二对角矩阵，将非平凡活动块按工作量排序后放入任务池。若任务数为 0，说明 GKH 已收敛；若任务数为 1，则根进程直接串行执行，避免把完整矩阵广播给其他进程却没有可用并行度；若任务数大于 1，则根进程广播当前 U, B, V ，工作进程领取活动块并执行一轮 bulge chasing。

Algorithm 1 MPI-GKH 活动块任务池

Input: 当前 U, B, V ，最大迭代次数 max_iter ，容忍误差 tol

Output: 收敛后的 U, S, V

```

1: for  $iter = 1$  to  $\text{max\_iter}$  do
2:   0 号进程清理数值噪声，并按超对角元素切分活动块
3:   将长度大于 1 的非平凡活动块放入任务池
4:   if 任务池为空 then
5:     广播停止命令，进入最终标准化
6:   else if 任务池只有一个活动块 then
7:     0 号进程本地执行该活动块，其他进程等待下一轮命令
8:   else
9:     0 号进程广播当前  $U, B, V$ 
10:    工作进程领取活动块并执行一轮 bulge chasing
11:    工作进程回传  $B[l:r, l:r]$ 、 $U[:, l:r]$ 、 $V[:, l:r]$ 
12:    0 号进程合并互不重叠的局部结果
13:   end if
14: end for
15: 0 号进程将奇异值非负化并降序排序
  
```

该设计没有强行并行化单个活动块内部的 Givens 旋转序列，因为 bulge chasing 本身存在顺序依赖。实验也会看到，进程数超过活动块并行度后，更多 MPI 进程不会自动带来加速。

2.4 通信量设计

在需要并行分发任务的一轮中，根进程广播完整的 U, B, V ，使所有工作进程从一致状态开始。工作进程完成活动块后不回传完整矩阵，而只回传：

$$B[l:r, l:r], \quad U[:, l:r], \quad V[:, l:r].$$

若活动块长度为 k ，矩阵规模近似为 $n \times n$ ，回传 double 数量约为 $k^2 + 2nk$ 。当活动块较小时，该策略比回传完整矩阵明显更省；当只有一个大块时，本实现退回根进程串行处理，避免没有并行收益的全矩阵广播。

3 实现与代码分析

3.1 文件结构

表1 给出本次实验涉及的主要文件。本报告关注算法、代码结构、实验数据和分析。

文件	作用
<code>matrix.h</code>	矩阵存储与基础运算。本次增加 <code>data()</code> 和 <code>size()</code> , 用于 MPI 广播连续内存。
<code>bidiagonalization.cpp</code>	Householder 上二对角化, 保留前序 SIMD 优化。
<code>gkh.cpp</code>	Golub-Kahan 迭代。本次新增 MPI 活动块任务池, 并保留 <code>serial</code> 、 <code>pthread</code> 、 <code>OpenMP</code> 路径。
<code>qsub_mpi.sh</code>	按修正脚本说明提交 MPI 作业。

表 1: 实验主要文件

3.2 矩阵连续内存接口

MPI 的 `MPI_Bcast` 只能广播连续缓冲区, 因此原有矩阵类需要暴露底层连续数组。这里没有改变矩阵数据结构, 只是在 `matrix.h` 中增加访问接口:

```
1 double *data() { return data_.data(); }
2 const double *data() const { return data_.data(); }
3 int size() const { return rows_ * cols_; }
```

这一改动的作用比较基础, 但很关键。若仍然通过逐行发送或逐元素发送矩阵, MPI 通信次数会大幅增加, 且代码容易在 U, B, V 三个矩阵同步时出错。使用连续内存后, 矩阵广播只需要先广播维度, 再广播 `double` 数组:

```
1 int dims[2] = {M.rows(), M.cols()};
2 MPI_Bcast(dims, 2, MPI_INT, 0, MPI_COMM_WORLD);
3 MPI_Bcast(M.data(), M.size(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

3.3 模式选择

MPI 模式选择放在 `gkh.cpp` 内部完成。程序读取环境变量 `SVD_MODE`; 如果显式设置为 `serial`、`pthread`、`openmp` 或 `mpi`, 则按指定模式运行; 若使用 `-DSVD_USE_MPI` 编译且没有设置环境变量, 则默认进入 MPI 路径。

```
1 #ifdef SVD_USE_MPI
2 return GkhMode::Mpi;
3 #else
4 return GkhMode::Pthread;
5 #endif
```

这样设计的好处是: 同一份 `main.cpp` 可以在普通 C++ 编译和 MPI 编译之间复用; 服务器上只需要改变编译命令, 而不需要改测试入口。MPI 初始化也采用延迟初始化: 当 GKH 第一次进入 MPI 路径时检查 `MPI_Initialized`, 若尚未初始化则由 `gkh.cpp` 调用 `MPI_Init`, 并通过 `atexit` 注册 `MPI_Finalize`。此外, 非 0 号进程会关闭 `std::cout`, 避免所有 rank 重复输出测试结果。

3.4 任务分发与工作进程协议

MPI 部分使用三类命令控制工作进程:

- `MPI_CMD_STOP`: GKH 已结束, 工作进程退出循环;
- `MPI_CMD_IDLE_SERIAL`: 本轮只有一个非平凡活动块, 由 0 号进程串行推进, 工作进程等待;
- `MPI_CMD_DISTRIBUTE`: 本轮存在多个活动块, 广播矩阵并分发任务。

根进程在任务池中按工作量从大到小排序活动块, 先给每个工作进程发送一个任务, 自己也处理一个任务。随后根进程使用 `MPI_ANY_SOURCE` 接收完成结果, 哪个进程先完成就继续给它分配下一个任务。这相当于 MPI 版的动态调度, 能够缓解活动块大小不均造成的负载不均。

```

1 MPI_Recv(header, 2, MPI_INT, MPI_ANY_SOURCE,
2         MPI_TAG_RESULT_HEADER, MPI_COMM_WORLD, &status);
3 const int worker = status.MPI_SOURCE;
4 ...
5 if (next_task < task_count) {
6     send_mpi_task(worker, tasks[next_task++]);
7 } else {
8     send_mpi_stop(worker);
9 }

```

这一协议也是后续实验结果的解释基础: 当 ready task 很少时, 动态调度没有足够任务可分, MPI 进程数再多也只能等待; 当 ready task 较多时, 任务池才有可能体现收益。

3.5 结果打包与通信开销

工作进程完成活动块 $[l, r]$ 后不回传完整矩阵, 而是把必要结果打包为一段连续数组。打包顺序为:

1. 活动块对应的 $B[l:r, l:r]$;
2. U 中第 l 到第 r 列;
3. V 中第 l 到第 r 列。

对于长度为 k 的活动块, 回传数据量为

$$k^2 + mk + nk$$

个 double。这比回传完整 U, B, V 小得多, 但前提是活动块足够小。另一方面, 每次进入并行分发轮次前, 根进程仍需广播完整 U, B, V , 这是当前实现最主要的通信成本。该设计是正确性优先的折中: 先保证所有工作进程从完全一致的矩阵状态出发, 再通过局部回传降低合并成本。后续若要继续优化, 最有价值的方向是减少广播范围, 例如只同步当前活动块相关列, 或让工作进程持有跨轮状态, 避免每轮全量同步。

3.6 统计字段与 profiling

`GkhRunStats` 中保留了迭代次数、活动块数量、可并行轮次、最大 ready task、MPI 任务发送数、矩阵广播字节数和结果回传字节数等字段。结合本次服务器数据, 可以确认主要瓶颈确实不在数值正确性, 而在活动块并行度与 MPI 通信开销的平衡。

4 实验方法

4.1 实验环境与数据采集

数据来自课程服务器，编译器使用 `mpic++`，MPI 头文件路径显示为 `mpich-aarch64`。任务通过 PBS 的 `qsub` 提交，每个计算节点最多申请 8 个核心，最多使用 4 个节点。

```
1 mpic++ -std=c++17 -O2 -DSVD_USE_MPI main.cpp bidiagonalization.cpp gkh.cpp -o
   main
2 qsub qsub_mpi.sh
```

实验固定随机种子为 2412592，采集了四类数据：MPI 规模扩展、编译优化等级、SIMD 开关、固定配置重复运行。所有结果均按类别保存为 `data/results/raw/**/test.o` 和 `test.e`，汇总表为 `data/results/summary.csv`。本报告基于 41 组有效运行数据进行统计，图中误差线为同组重复运行的标准差。

图表绘制使用 Python 对 `summary.csv` 做二次统计，并采用 `SciencePlots` 与 `Matplotlib` 生成。增加了误差线、关键点标注、堆叠时间构成和热力图汇总，便于直接观察最优进程数、负优化区间和不同优化手段的贡献。

MPI 初始化放在 `gkh.cpp` 的 GKH 入口中。每个 MPI 进程在进入 GKH 前都会执行一次主程序中的上二对角化阶段，`test.o` 中也会看到若干重复样例标题。真正的 MPI 并行优化对象是 GKH 阶段，所以本文在讨论规模扩展时以 GKH 时间为主，同时保留端到端总时间作为参考。

4.2 Profiling 设计

本文采用“服务器计时数据 + 本地热点采样”的方式完成分析。服务器端主要依据 41 组真实 PBS 运行输出中的上二对角化时间、GKH 时间、不同进程数下的均值和标准差。这样可以直接观察 MPI 进程数增加后，通信、复制、等待和任务粒度对端到端时间的影响。作为补充，本地使用 Intel VTune 对 `main.cpp` 单进程入口做热点采样，用来判断子进程内部主要计算热点。

5 实验结果与分析

5.1 正确性验证

41 组数据全部显示 通过：5 / 5，未出现 FAIL 或 `converged: no`。所有测试样例中，最大相对重构误差为 4.80×10^{-13} ，最大 U 正交误差为 2.35×10^{-13} ，最大 V 正交误差为 2.38×10^{-13} 。这说明 MPI 任务分发、局部结果合并和 SIMD 开关都没有破坏 SVD 数值正确性。

指标	最大值	典型量级	结论
相对重构误差 $\ A - USV^T\ _F / (\ A\ _F + 1)$	4.80×10^{-13}	10^{-13}	通过
$\ U^T U - I\ _F$	2.35×10^{-13}	10^{-13}	通过
$\ V^T V - I\ _F$	2.38×10^{-13}	10^{-13}	通过
通过样例数	5/5	所有 41 组	通过

表 2: 正确性实验摘要

5.2 MPI 规模扩展

表3 与图1 展示了不同进程数下的平均耗时。GKH 阶段在 $np = 4$ 时达到本组实验最好结果，平均耗时为 15975.63 ms，相对 $np = 1$ 的 23147.60 ms 加速 1.45 倍；但是 $np = 8$ 时回升至 17322.33 ms， $np = 16$ 时进一步回升至 19177.90 ms， $np = 32$ 时显著退化为 67490.93 ms。端到端总时间的最优点同样出现在 $np = 4$ ，为 22707.15 ms，相对 $np = 1$ 的 27481.56 ms 加速 1.21 倍。

np	nodes	ppn	上二对角/ms	GKH/ms	总时间/ms	GKH 加速比	GKH 效率
1	1	1	4333.96	23147.60	27481.56	1.00	100.0%
2	1	2	6080.27	22535.20	28615.47	1.03	51.4%
4	1	4	6731.51	15975.63	22707.15	1.45	36.2%
8	1	8	8795.52	17322.33	26117.86	1.34	16.7%
16	2	8	9580.87	19177.90	28758.77	1.21	7.5%
32	4	8	9400.84	67490.93	76891.77	0.34	1.1%

表 3: MPI 规模扩展实验平均结果，固定种子 2412592，每组 3 次

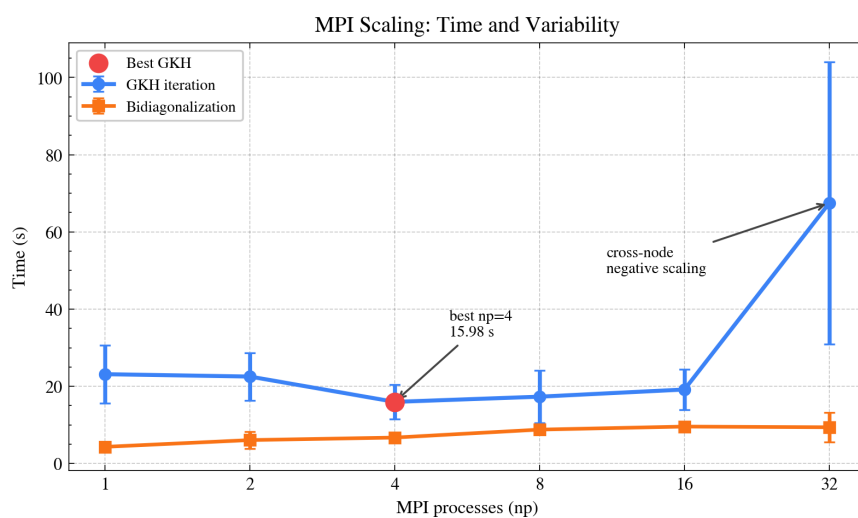


图 1: MPI 规模扩展：上二对角化与 GKH 平均耗时

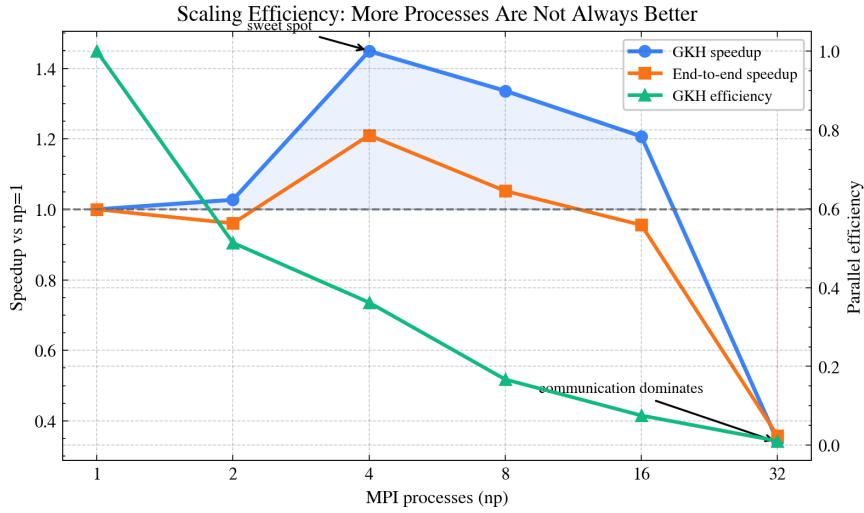


图 2: MPI 规模扩展: 加速比与并行效率

图3 用热力图汇总了同一组数据。所有行均为“越高越好”的指标，可以直观看出 $np = 4$ 同时具有最高 GKH 加速比和最高端到端加速比，而 $np = 32$ 在所有性能指标上都明显退化。尤其是 GKH 并行效率从 $np = 4$ 的 36.2% 下降到 $np = 8$ 的 16.7%、 $np = 16$ 的 7.5%，到 $np = 32$ 只剩 1.1%。这表明瓶颈不是计算核心数量不足，而是可以被分发的有效任务太少。

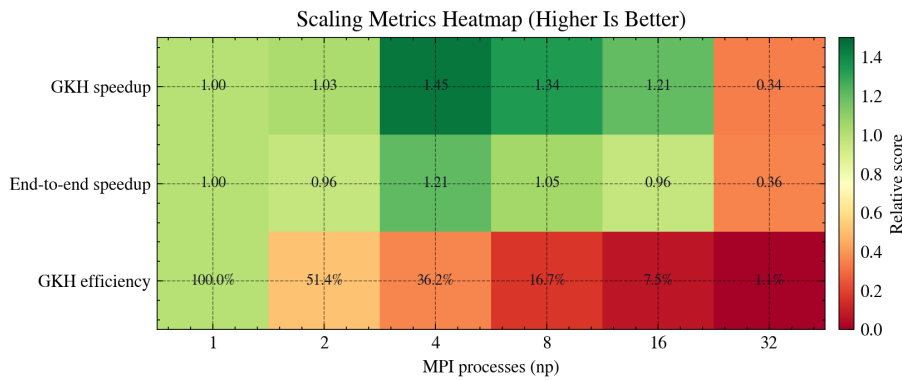


图 3: 规模扩展关键指标热力图

该结果说明，当前端到端随机矩阵测试中的可用活动块数量不足以支撑大量 MPI 进程。 $np = 2$ 只有 1.03 倍 GKH 加速，说明任务池已经被同步和调度开销明显抵消； $np = 4$ 是较平衡点，仍在单节点内，通信成本较低； $np = 16$ 进入跨节点后，广播 U, B, V 和回收局部结果的代价上升； $np = 32$ 虽满足脚本上限，但进程数远大于可同时处理的活动块数量，通信和等待成为主要开销，因此出现负优化。这也与实验手册中“大于 32 进程后通信时间会显著增长、加速意义不明显”的提示一致。

从数据波动看， $np = 32$ 的 GKH 标准差约为 36.5 秒，明显高于其他组。这说明进程数过多不只是平均速度变慢，也会让运行稳定性变差。原因可能包括跨节点分配差异、每轮广播矩阵时网络状态变化，以及少量活动块在 32 个进程之间造成更严重的等待放大。相比之下， $np = 4$ 的 GKH 标准差约为 4.5 秒，虽然仍受服务器负载影响，但整体更加可控。

图4 从端到端角度展示了时间构成。上二对角化没有被 MPI 分布式化，且会在各 rank 中重复执行，因此它并不随 np 增加而下降，反而可能因为多进程同时运行和节点负载变化而波动。即便如此，GKH 仍然占主要部分，说明将优化重点放在 GKH 阶段是合理的。也正因为上二对

角化不是本次 MPI 优化对象，本文在判断并行策略时优先观察 GKH 时间和 GKH 加速比，而把端到端时间作为整体工程代价。

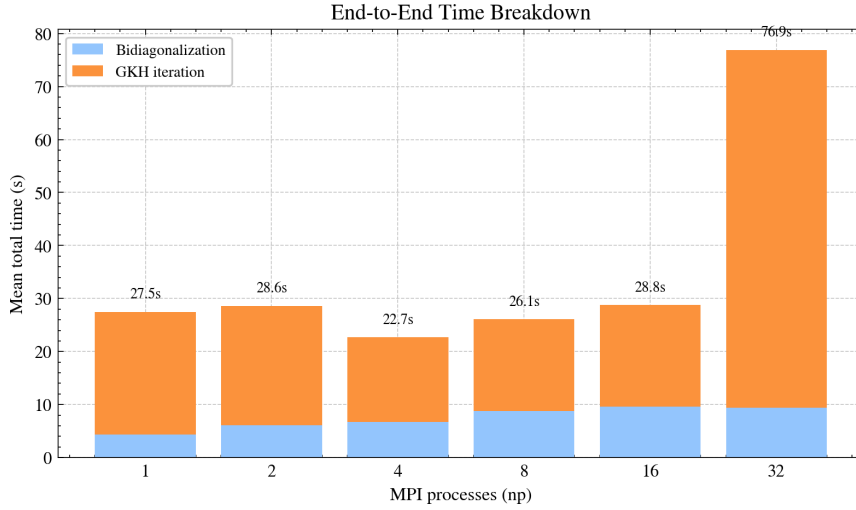


图 4: 端到端时间构成

5.3 基于计时数据的开销分析

对 1000×1000 主测试样例而言, U, B, V 三个矩阵各约含 10^6 个 `double`, 单矩阵约 8 MB, 三者合计约 24 MB。由于 MPI 进程内存空间相互独立, 每个 rank 都需要持有自己的矩阵副本; 当进入并行分发轮次时, 根进程还需要广播当前 U, B, V , 这正是多进程 SVD 比多线程更难加速的核心原因之一。

结果回传虽然只传活动块相关数据, 长度为 k 的活动块回传量约为 $k^2 + mk + nk$ 个 `double`, 比完整矩阵小得多, 但这种优化只有在活动块已经切分得比较小时才明显有效。如果迭代中长期只有一个大的非 1×1 活动块, 任务池没有足够任务可分, 程序会退回 0 号进程串行推进; 如果强行让所有 worker 参与, 则每轮广播 24 MB 级矩阵却只能处理少量有效任务, 通信与复制成本会盖过计算收益。

表3 给出了这一现象的数据证据: $np = 4$ 时 GKH 平均时间为 15975.63 ms, 取得 1.45 倍 GKH 加速; 继续增至 $np = 8$ 后, GKH 时间回升到 17322.33 ms; 跨节点 $np = 16$ 为 19177.90 ms; $np = 32$ 更退化到 67490.93 ms。若瓶颈只是计算量不足, 增加进程数至少不应导致如此明显的退化; 实际结果说明, 复制、广播、结果合并和进程等待已经成为主要开销。 $np = 32$ 的 GKH 标准差约 36.5 秒, 也说明高进程数下性能不仅更慢, 而且更不稳定。

因此, 本文的 profiling 结论是: 当前实现中真正有收益的是中等进程数下对多个活动块的动态调度; 当活动块数量不足或跨节点通信放大时, 进程独立内存和全量矩阵同步会迅速抵消收益。

5.4 编译优化等级影响

表4 和图5 比较了 -O0、-O1、-O2、-O3。在固定 $np = 8$ 和启用 SIMD 的情况下, -O0 总时间为 121703.03 ms; -O1、-O2、-O3 均将总时间降到约 25 秒, 其中 -O2 最快, 总加速比为 4.82。

优化等级	上二对角/ms	GKH/ms	总时间/ms	相对 O0 加速比
-00	26995.80	94707.23	121703.03	1.00
-01	6727.19	19179.70	25906.89	4.70
-02	6203.41	19043.00	25246.41	4.82
-03	6030.14	19347.33	25377.48	4.80

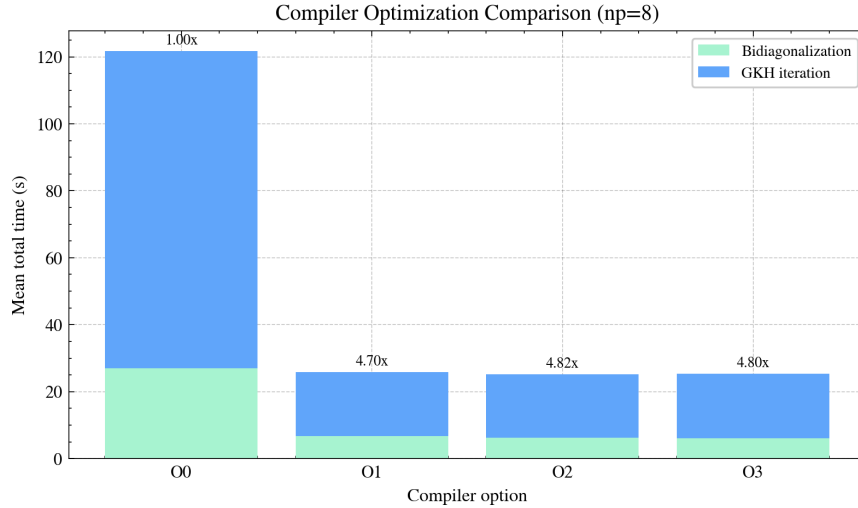
表 4: 编译优化等级实验, 固定 $np = 8$ 、SIMD、种子 2412592, 每组 3 次

图 5: 编译优化等级对端到端耗时的影响

优化等级实验说明, 编译器优化是本程序的基础性能条件。没有优化时, GKH 中大量循环、Givens 旋转和矩阵列更新会产生巨大开销; 开启 -01 后大部分收益已经出现, GKH 时间从 94707.23 ms 降到 19179.70 ms, 约为原来的 20.3%。-02 相比 -01 只继续减少约 3.5% 的总时间, -03 反而略慢于 -02。这说明当前瓶颈并不只是单核浮点计算, 调度、通信和内存访问也在限制性能。

这组数据还提醒我们不能把编译器优化收益误认为 MPI 收益。如果把 -00 的结果和 -02 的 MPI 结果直接比较, 会得到非常大的“加速比”, 但那主要来自编译器优化而不是并行任务池。因此后续规模扩展和 SIMD 对比均固定在 -02 下进行。

5.5 SIMD 与 MPI 的组合

表5 和图6 给出 SIMD 开关对比。启用 SIMD 后, 上二对角化平均耗时从 10367.03 ms 降至 8232.26 ms, 加速约 1.26 倍; 但 GKH 平均耗时从 20227.73 ms 变为 20337.07 ms, 几乎没有提升, 甚至略慢; 总时间加速约 1.07 倍。

SIMD 设置	上二对角/ms	GKH/ms	总时间/ms	总时间标准差/ms
启用 SIMD	8232.26	20337.07	28569.33	2922.36
禁用 SIMD	10367.03	20227.73	30594.76	2553.58

表 5: SIMD 开关对比, 固定 $np = 8$ 、-02、种子 2412592, 每组 3 次

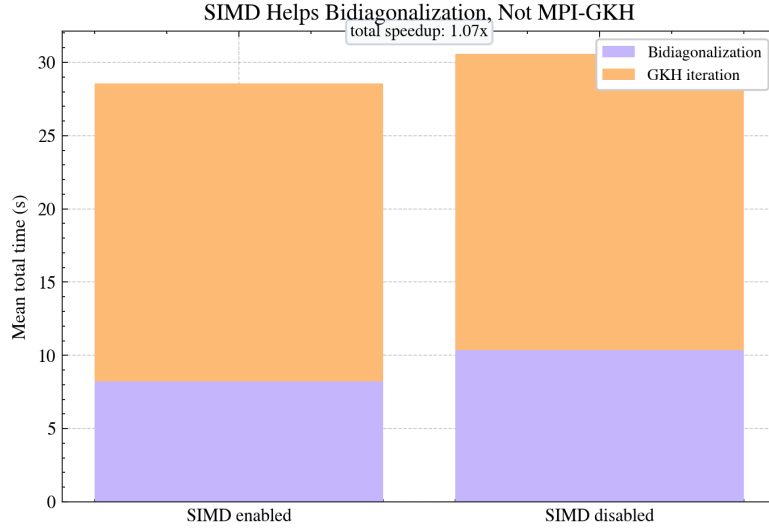


图 6: SIMD 开关对比

这组数据说明：MPI+SIMD 不一定优于只优化 SIMD，也不一定让 GKH 阶段明显变快。SIMD 主要加速连续向量运算，例如范数、点积和批量列更新；而本次 MPI-GKH 中，普通随机矩阵的瓶颈更多来自活动块并行度不足、矩阵广播、进程等待和结果合并。上二对角化阶段有较多连续向量计算，所以启用 SIMD 后从 10367.03 ms 降到 8232.26 ms；GKH 阶段虽然也有局部向量更新，但每轮还包含活动块扫描、shift 计算、Givens 参数生成和 MPI 同步，SIMD 覆盖比例较低，因此 20337.07 ms 与禁用 SIMD 的 20227.73 ms 基本处于实验波动范围内。

换言之，SIMD 优化提高的是单进程局部计算效率，而 MPI 任务池解决的是活动块级任务分发。两者不是简单相乘关系：当任务粒度足够大且通信相对较少时，SIMD 与 MPI 可以叠加；当通信和等待占主导时，SIMD 的局部收益会被更高层的并行开销吞没。

5.6 重复运行稳定性

为避免单次测量受队列状态和节点负载影响，固定 -O2+SIMD、 $np = 8$ 、种子 2412592 额外重复运行 5 次。结果见表 6 和图 7。总时间均值为 26476.07 ms，标准差为 2110.68 ms，变异系数为 8.0%；GKH 变异系数为 9.3%。

重复次数	上二对角/ms	GKH/ms	总时间变异系数
5	6673.45 ± 1354.86	19802.62 ± 1831.75	8.0%

表 6: 固定配置重复实验，-O2+SIMD、 $np = 8$ 、种子 2412592

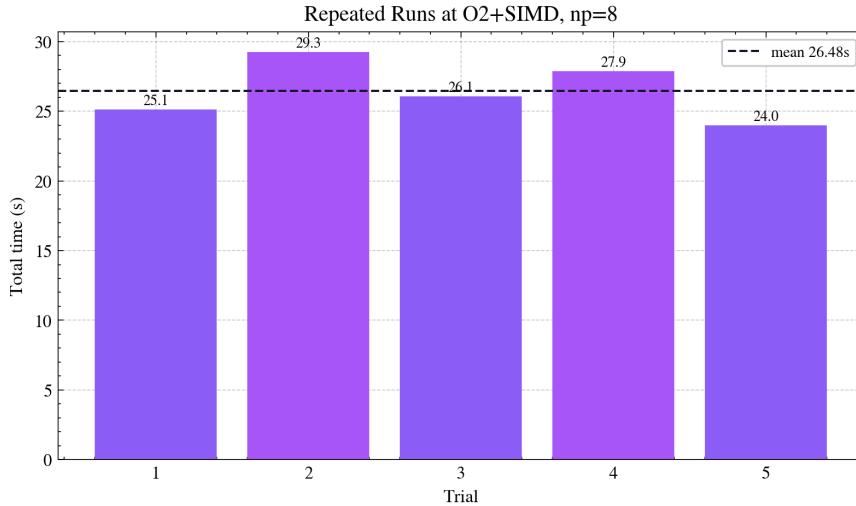


图 7: 固定配置重复运行的总时间波动

这说明服务器实验存在一定运行波动，尤其是 MPI 作业会受到节点分配、队列负载和跨节点通信状态影响。因此本文对规模扩展、优化等级和 SIMD 开关均使用多次重复的平均值，而不是选择单次最优结果。固定 $np = 8$ 的重复实验中，总时间变异系数为 8.0%，GKH 时间变异系数为 9.3%，属于可以接受但不能忽略的波动范围。该处理也解释了部分表格中标准差较大的现象，例如 $np = 32$ 的 GKH 标准差达到约 36.5 秒，说明过多进程不仅平均性能差，稳定性也更差。

因此，在评价本实验时，更有意义的不是某一次运行是否偶然较快，而是多次平均后是否呈现稳定趋势。规模扩展实验的趋势非常清楚：从 $np = 1$ 到 $np = 4$ 有有限加速，从 $np = 8$ 开始收益下降，跨节点和高进程数下出现负优化。这一趋势与 SVD-GKH 活动块并行度有限的算法分析相一致。

5.7 本地 VTune 热点 profiling

在本地 Windows 环境中，使用 VTune 对 -O2、启用 SIMD、固定种子 2412592 的 `main.cpp` 单进程运行进行热点采样。该 profiling 不用于替代服务器 MPI 实验，因为它没有跨进程通信；它的作用是回答“每个进程内部主要把 CPU 时间花在哪里”。本地运行 5 个样例全部通过，最大规模 1000x1000 样例中，上二对角化耗时 1935.63 ms，GKH 耗时 4129.38 ms，总 GKH 耗时 4239.91 ms。

热点函数	CPU 时间/s	占比	解释
<code>apply_right_cols_range</code>	2.689	36.3%	GKH 中对列区间应用右乘 Givens 更新
<code>Matrix::operator*</code>	1.405	18.9%	<code>main.cpp</code> 正确性验证中的矩阵重构乘法
<code>cleanup_bidiagonal</code>	0.618	8.3%	清理近零超对角元并划分活动块
<code>_mm_add_pd</code>	0.400	5.4%	SIMD 双精度加法指令
<code>_mm_loadu_pd</code>	0.354	4.8%	SIMD 非对齐加载指令
<code>simd_dot</code>	0.275	3.7%	上二对角化中的向量点积

表 7: 本地 VTune hotspots 结果，官方 `main.cpp` 单进程运行

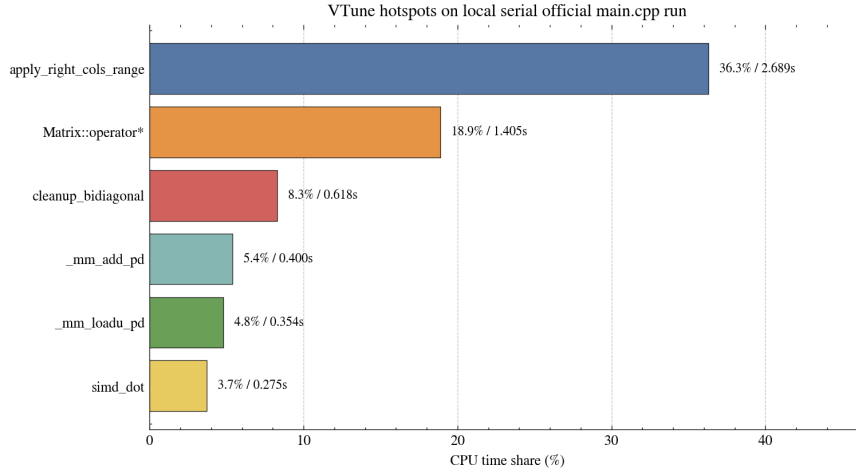


图 8: 本地 VTune 热点函数占比

表7和图8表明,子进程内部最主要的热点是 GKH 的列区间更新函数 `apply_right_cols_range`, 占 CPU 时间 36.3%。这与算法分析一致: bulge chasing 每一步都需要对 B 和 V 的相关列做 Givens 旋转, 虽然局部操作可以被 SIMD 化, 但访问模式和迭代依赖仍然决定了它是 GKH 的主要计算核。第二个热点 `Matrix::operator*` 来自 `main.cpp` 中的正确性验证, 即计算 USV^T 的重构误差; 它不属于被优化的 GKH 算法本体, 但会出现在端到端时间和本地 profiling 中。

这一结果可以解释 SIMD 实验中的现象: VTune 的热点表中确实出现了 `_mm_add_pd`、`_mm_loadu_pd`、`simd_dot` 等 SIMD 相关项, 说明向量化代码被执行; 但是在 MPI+GKH 场景中, 活动块划分、清理、矩阵同步和任务等待共同参与耗时, 所以 SIMD 指令热点并不能直接转化为 GKH 阶段的线性加速。也就是说, 子进程内部仍有计算热点可优化, 但服务器规模扩展曲线的主要限制已经上升到活动块并行度和 MPI 通信层面。

6 进阶要求探索

6.1 不同平台与并行形态对比

本实验主要运行平台是课程服务器, 编译环境显示 MPI 头文件来自 `mpich-aarch64`, 因此服务器数据可视为 ARM 平台上的 MPI+SIMD 实验。作为对照, 本地 Windows x86 环境中保留了 serial、pthread 与 VTune profiling 数据。两类平台的 CPU、编译器、系统负载和矩阵规模并不完全一致, 所以本文不把它们绝对耗时直接横向比较, 而只比较“同一平台、同一输入内部”的加速趋势。

平台	并行形态	代表样例	GKH/ms	加速比	可并行轮次
x86 本地	serial+SIMD	随机 512×512	4421.59	1.00	—
x86 本地	pthread4+SIMD	随机 512×512	4402.29	1.00	8/749
x86 本地	OpenMP4+SIMD	随机 512×512	4385.86	1.01	8/749
x86 本地	serial+SIMD	4 个 128 块	1130.35	1.00	—
x86 本地	pthread4+SIMD	4 个 128 块	360.63	3.13	252/253
x86 本地	OpenMP4+SIMD	4 个 128 块	359.08	3.15	252/253
ARM 服务器	MPI+SIMD, $np = 1$	随机 1000×1000	23147.60	1.00	未输出
ARM 服务器	MPI+SIMD, $np = 4$	随机 1000×1000	15975.63	1.45	未输出
ARM 服务器	MPI+SIMD, $np = 32$	随机 1000×1000	67490.93	0.34	未输出

表 8: 不同平台和并行形态的实测数据对比

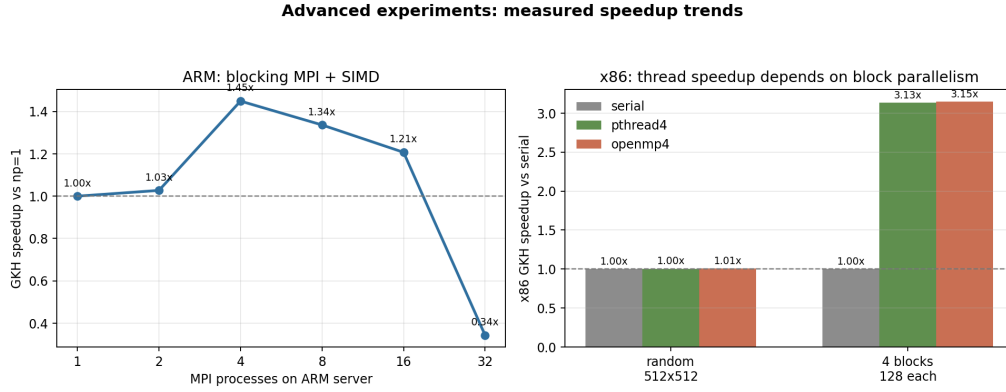


图 9: 不同平台与并行形态的相对加速趋势

本地补充实测在 Windows x86_64 环境中完成, 编译器为 MinGW-W64 g++ 7.3.0。这组实验不用于和 ARM 服务器做绝对耗时排名, 而是固定同一平台、同一输入, 比较 serial、pthread4 与 OpenMP4 在活动块并行度不同的情况下能否加速。OpenMP4 版本的编译与运行命令如下, 随机种子固定为 20260408:

```

1 g++ -std=c++17 -O2 -fopenmp experiment_driver.cpp \
2     bidiagonalization.cpp gkh.cpp \
3     -o profiling/experiment_openmp.exe
4
5 $env:SVD_MODE='openmp'
6 $env:SVD_THREADS='4'
7 .\profiling\experiment_openmp.exe --seed 20260408 --repeat 1 \
8     --sizes 512x512 --gkh-blocks 128x4

```

实测输出中两个样例均为 PASS。随机 512×512 的相对重构误差为 3.7935×10^{-13} , OpenMP4 的 GKH 时间为 4385.86 ms; 4 个 128 块结构样例的相对重构误差为 3.8720×10^{-13} , OpenMP4 的 GKH 时间为 359.08 ms。与同一输入下的 serial 和 pthread4 数据放在一起可以看到: 随机矩阵中 OpenMP4 只比 serial 快约 1.01 倍, 而块结构中 OpenMP4 可达到 3.15 倍。这说明本地实测的核心变量不是“线程 API 名称”, 而是活动块是否真的提供了足够的并行任务。

Advanced experiments: parallel opportunity evidence

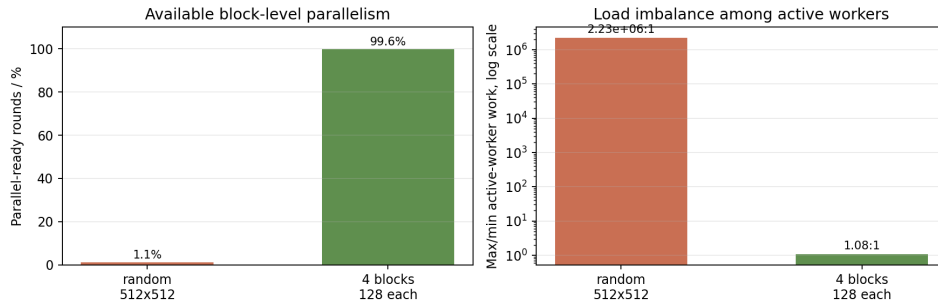


图 10: 活动块并行机会与工作量不均衡

表8、图9 和图10 的关键结论是: 平台差异不是唯一决定因素, 活动块并行度更直接决定了并行收益。本地 x86 上, 随机 512×512 的可并行轮次只有 8/749, pthread4 和 OpenMP4 的 GKH

加速比分别只有 1.00 和 1.01；同时 active worker 的最大/最小工作量比例约为 $2.23 \times 10^6 : 1$ ，说明几乎所有实质工作仍集中在一个 worker 上。相反，4 个 128 块样例中可并行轮次为 252/253，最大/最小工作量比例仅约 1.08 : 1，pthread4 和 OpenMP4 分别达到 3.13 和 3.15 倍加速。ARM 服务器上也呈现类似趋势：当 $np = 4$ 仍能承接一定活动块时，MPI+SIMD 有有限收益；当 $np = 32$ 远大于可同时处理的活动块数量时，通信和等待开销超过计算收益，性能显著退化。

这组进阶对比说明，多线程、SIMD 和 MPI 的关系不是简单叠加。SIMD 改善单个 rank 或线程内部的向量运算，多线程和 MPI 需要足够多的独立任务才能发挥作用。若算法长期只有一个大活动块，换成更多线程、更多进程或更强平台都难以自动获得线性加速。

6.2 不同 MPI 编程方法的比较

本次最终代码采用阻塞双边通信：根进程通过 MPI_Bcast 同步矩阵状态，通过 MPI_Send/MPI_Recv 下发任务和回收结果，并使用 MPI_ANY_SOURCE 实现动态任务领取。需要说明的是，本文没有把 MPI_Isend/MPI_Irecv、RMA 单边通信分别实现为独立版本，因此不把它们写成“实测加速对比”。本节的比较方式是：以已经实测的阻塞 MPI 规模扩展数据、x86 线程并行机会数据和通信量估算，判断这些方法在本 SVD-GKH 场景中的潜在收益和风险。表9 给出具体数据依据。

方法	具体实验数据支撑	判断
阻塞双边通信	已实现并完成 41 组服务器实验，全部通过：5 / 5。规模扩展中 $np = 4$ 的 GKH 平均耗时为 15975.63 ms，加速比 1.45； $np = 32$ 的 GKH 平均耗时退化到 67490.93 ms，加速比 0.34，标准差 36518.19 ms。	阻塞通信正确性最好，也能在 $np = 4$ 取得有限收益；但高进程数下通信、等待和节点波动会迅速吞掉收益。
非阻塞双边通信	x86 本地随机 512×512 的可并行轮次只有 8/749 = 1.07%，最大 ready task 只有 2；同一输入下 pthread4 和 OpenMP4 加速比分别只有 1.00 和 1.01。相反，4 个 128 块样例的可并行轮次为 252/253 = 99.60%，pthread4/OpenMP4 可达到 3.13/3.15 倍。	非阻塞通信只有在 ready task 足够多时才有较大重叠空间。普通随机矩阵中可重叠计算窗口太小，MPI_Isend/MPI_Irecv 预计只能隐藏少量回传等待；块结构输入更值得尝试。
单边通信 RMA	对 1000×1000 主样例， $U + B + V$ 完整矩阵约 24 MB；阻塞方案在 $np = 32$ 时单轮完整广播量估计为 744 MB。随机矩阵又只有 1.07% 的可并行轮次，说明大多数轮次没有足够 worker 可以共享窗口。	RMA 不能自动减少需要同步的数据量；若仍同步完整矩阵，通信量瓶颈仍在。只有改成活动块相关列的窗口，并解决 U, V 列更新一致性问题上，单边通信才可能有收益。
MPI 自身多线程支持	本地 x86 实测显示：随机 512×512 下 pthread4/OpenMP4 基本不加速；4 个 128 块结构下，pthread4 为 360.63 ms、OpenMP4 为 359.08 ms，分别相对 serial 的 1130.35 ms 达到 3.13/3.15 倍。块结构中 worker 最大/最小工作量比例仅 1.08:1。	混合并行是更有数据支撑的后续方向：节点间减少 MPI rank，节点内使用线程池处理活动块，可减少矩阵副本和广播参与者，同时保留块结构下的线程级收益。

表 9: 不同 MPI 编程方法的数据支撑与适用性判断

np	每轮完整 $U + B + V$ 广播量/MB	GKH 平均耗时/ms	GKH 加速比
1	0	23147.60	1.00
2	24	22535.20	1.03
4	72	15975.63	1.45
8	168	17322.33	1.34
16	360	19177.90	1.21
32	744	67490.93	0.34

表 10: 阻塞 MPI 方案的通信量估算与实测性能

从当前 41 组服务器数据和表10 看，阻塞 MPI 的收益与通信量之间存在明显转折：从 $np = 1$

到 $np = 4$ ，每轮完整矩阵广播量从 0 MB 增至 72 MB，但 GKH 平均耗时从 23147.60 ms 降至 15975.63 ms，说明此时仍有足够任务收益抵消通信代价；继续增至 $np = 8$ 和 $np = 16$ 后，广播量估计增至 168 MB 和 360 MB，GKH 加速比分别下降到 1.34 和 1.21；到 $np = 32$ 时，单轮广播量估计达到 744 MB，GKH 平均耗时反而退化到 67490.93 ms，且标准差达到 36518.19 ms。这个数据趋势说明，高进程数下的主要问题不是 MPI_Send/MPI_Recv 是否写成非阻塞形式，而是需要同步的数据量和可并行任务数之间已经失衡。

因此，最值得继续尝试的不是盲目引入更复杂的通信模型，而是混合并行：节点间使用较少 MPI rank，节点内复用已有 pthread/OpenMP 活动块任务池。这个判断同样有本地实测支撑。图10 中，普通随机矩阵只有 1.07% 的可并行轮次，pthread4/OpenMP4 都约为 1.0 倍；而块结构样例有 99.60% 的可并行轮次，pthread4/OpenMP4 可达到 3.13/3.15 倍。这说明非阻塞通信和单边通信在普通随机矩阵上能重叠的计算窗口很小，更适合作为有块结构输入或混合并行方案上的后续优化分支。

6.3 其他算法优化策略

除通信方式外，本实验还可以从算法层面继续优化。第一，设置任务粒度阈值：当活动块数量少或块太小时，直接在根进程串行推进，避免广播完整矩阵。当前实现已经包含“只有一个非平凡块时串行推进”的思想，后续可把阈值从数量扩展到估算工作量。第二，减少同步数据量：不要每轮广播完整 U, B, V ，而是只同步与活动块相关的 B 子块和 U, V 列区间；这一策略能够直接降低表9 中阻塞通信的主要开销。第三，延迟更新正交矩阵：先记录 Givens 旋转序列，待若干轮后批量更新 U, V ，可能提升缓存局部性和 SIMD 覆盖率，但需要额外验证重构误差和数值稳定性。第四，构造并识别块结构输入：本地 x86 的 4 块样例说明，只要活动块数量充足，已有任务池就能取得明显收益；因此对具有块结构或近似块对角结构的矩阵，MPI 任务池更可能发挥作用。

7 综合讨论

7.1 为什么 $np = 4$ 最优

从实验结果看， $np = 4$ 是当前实现和测试数据上的最佳折中。它仍然位于单节点内，通信主要是节点内 MPI 通信，广播和回收开销较低；同时相比 $np = 1$ 和 $np = 2$ ，它又有足够进程承接 deflation 后出现的多个活动块。继续增加到 $np = 8$ 后，并行度增长已经赶不上调度和等待成本；跨节点的 $np = 16$ 、 $np = 32$ 则进一步放大了数据复制和通信开销。

这说明本实验的性能瓶颈不是“没有 MPI”，而是 SVD-GKH 本身很难在普通随机矩阵上长期提供大量独立任务。实验手册中的特殊情况说明也指出，迭代中可能长期只有一个非 1×1 大矩阵，其余矩阵都已经收敛；本实验的规模扩展曲线正体现了这一性质。

7.2 MPI 任务池的价值与限制

任务池的价值在于正确处理了负载不均衡问题：当多个活动块存在时，工作进程不是静态绑定某个块，而是空闲后继续领取任务。这比固定块划分更适合 GKH 中块大小不断变化的过程。但任务池不能创造算法并行度。如果某一轮只有一个大活动块，根进程串行推进反而是更优选择，因为强行广播完整矩阵只会增加开销。

因此，当前实现对“块数量足够”的场景是有效的，对普通随机矩阵只能获得有限收益。后续如果要进一步优化，可以考虑两条方向：一是寻找活动块内部更细粒度但数学上安全的并行更

新；二是减少广播量，例如只同步与当前活动块相关的矩阵列，或在多轮任务之间复用工作进程状态，避免每次都广播完整 U, B, V 。

7.3 与前序 SIMD 实验的关系

SIMD 优化仍然是必要的基础优化。它对上二对角化阶段有稳定收益，并能降低局部向量运算成本。但在 MPI-GKH 中，SIMD 无法解决进程等待、活动块数量不足和通信放大问题。因此不能简单地说“MPI+SIMD 一定最快”。更准确的结论是：SIMD 提高单进程计算核效率，MPI 提供跨进程任务调度能力；只有当任务粒度足够大、通信足够少、活动块足够多时，二者叠加才可能得到明显收益。

8 总结

本次实验完成了 SVD 算子的 MPI 并行化实现和服务端实验验证。代码层面将 MPI 初始化和任务池逻辑封装在 `gkh.cpp` 中；实验流程层面按照修正脚本说明，使用 `mpic++` 编译和 `qsub_mpi.sh` 提交。

从 41 组真实服务器数据看，程序正确性稳定，全部测试通过，误差保持在 10^{-13} 量级。性能方面，GKH 阶段在 $np = 4$ 时取得最好平均结果，加速比为 1.45； $np = 8$ 后收益下降， $np = 32$ 出现明显负优化。编译优化对性能影响显著，-O2 相比 -O0 端到端加速 4.82 倍；SIMD 对上二对角化有 1.26 倍收益，但对 MPI-GKH 基本无提升。综合来看，SVD 的 MPI 加速受活动块并行度强烈限制，任务池实现是必要的，但通信和算法粒度决定了其上限。

进阶探索部分进一步对比了 ARM 服务器 MPI+SIMD 与本地 x86 pthread/OpenMP+SIMD 的趋势。结果表明，随机矩阵的可并行轮次只有 8/749，pthread4 和 OpenMP4 都只能获得约 1.0 倍 GKH 加速；人工块结构样例的可并行轮次达到 252/253，pthread4 与 OpenMP4 分别达到 3.13 和 3.15 倍 GKH 加速。MPI 通信量估算也显示， $np = 32$ 时完整 $U + B + V$ 广播量每轮可达约 744 MB。这说明后续优化的重点应放在减少通信复制、提高活动块并行度以及发展 MPI+ 线程的混合并行，而不是单纯增加进程数。

附录 A：生成式 AI 辅助学习对话记录

本附录记录了实验过程中利用生成式 AI 辅助理解 SIMD、多线程和 MPI 编程的对话截图。提问方式遵循“先给出自己的实现、实验数据或瓶颈判断，再请求 AI 检查和分析”的原则，避免直接要求 AI 代写完整代码或编造实验结论。



图 11: AI 辅助学习对话记录 1

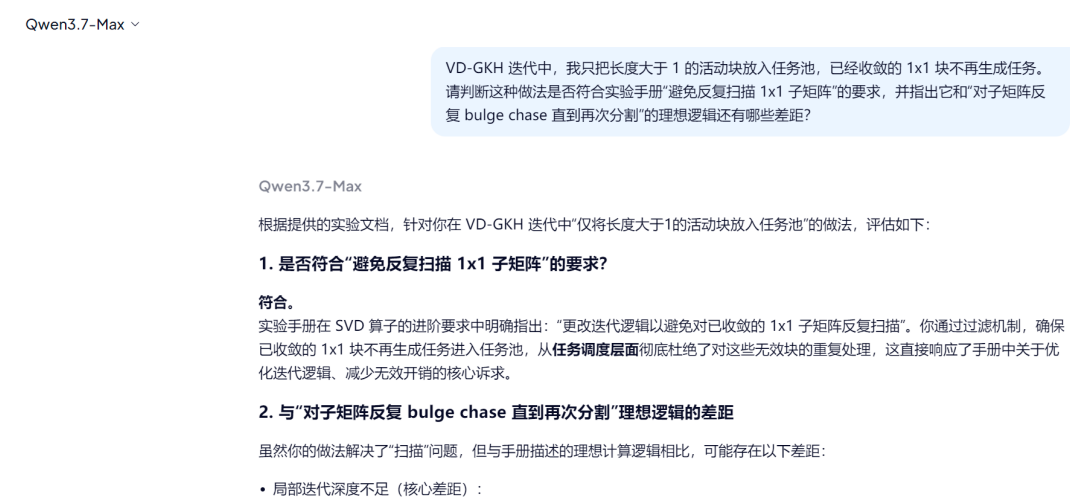


图 12: AI 辅助学习对话记录 2



图 13: AI 辅助学习对话记录 3

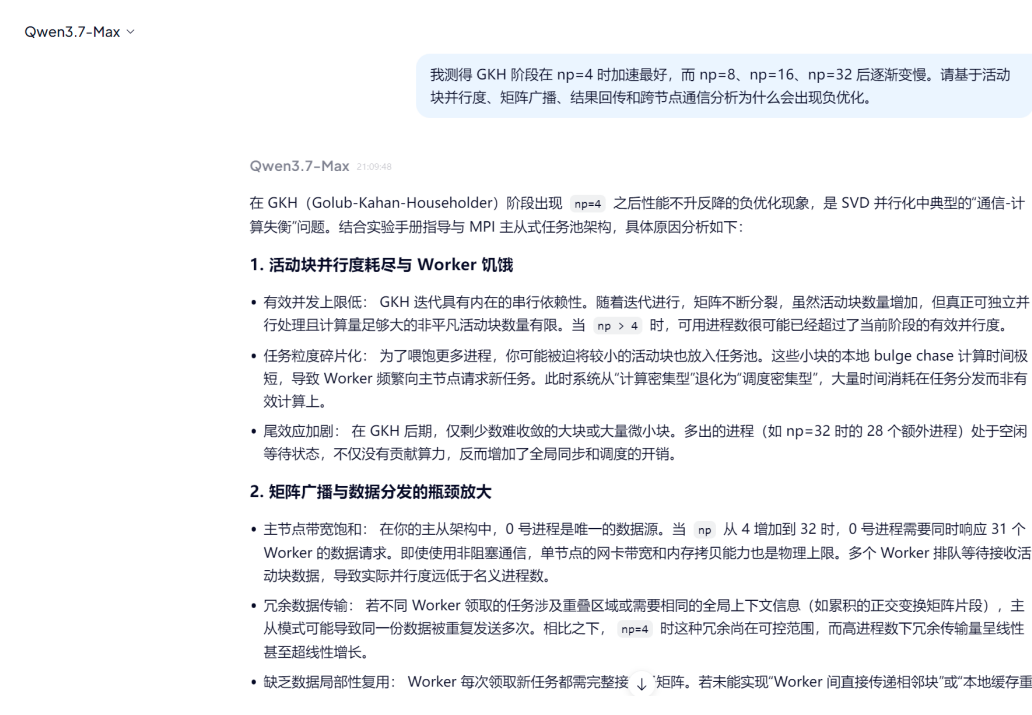


图 14: AI 辅助学习对话记录 4

参考文献

- [1] Gene H. Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965.
- [2] Gene H. Golub and Christian Reinsch. Singular value decomposition and least squares solutions. *Numerische Mathematik*, 14:403–420, 1970.